

II.2 Genetic Algorithms

Genetic algorithms are inspired by principles of genetics. They have applications in many types

We consider the application to the maximization problem:

$$\begin{array}{ll} \text{maximize} & f(x) \\ \text{subject to} & x \in \Omega \end{array}$$

(A minimization problem for $f(x)$ can always be changed to a maximization problem by maximizing $-f(x)$.)

In maximization, the objective function $f(x)$ is often called **fitness function**.

Idea: 1. Start with an initial set of points in Ω , denoted $P(0)$, called the *initial population*.

2. Evaluate the objective function $f(x)$ at all the points in the set $P(0)$. Based on this evaluation, create a new set of points $P(1)$ through some operations called *crossover* and *mutation* (to be explained later). The average of the values of $f(x)$ at these new points should be higher than the average of the values at the old points.)

3. Repeat step 2, generating populations , $P(2)$, $P(3)$,... until a stopping criterion is reached.

a) The basic genetic algorithm.

We explain the algorithm by the following example:

$$\text{Maximize} \quad f(x) = (x - 10)^2$$

subject to: x is an integer between 0 and $63 = 2^6 - 1$

We start by *encoding* the points of $\Omega = \{ 0, 1, 2, 3, \dots, 63 \}$ as strings of equal length,

e.g. 39 is encoded as $1\ 0\ 0\ 1\ 1\ 1$

Such a string is called a *chromosome* of length $L = 6$, and the set of symbols $\{ 0, 1 \}$ composing the string is called an *alphabet*.

(In the above example, the encoding of 39 is simply its binary representation, but other encodings and other alphabets are possible. The choice of *chromosome length*, of *alphabet* and *encoding* is called a *representation scheme*.

For each chromosome $\mathbf{x}$, we call $f(\mathbf{x})$ the *fitness* of $\mathbf{x}$.

e.g. the fitness of chromosome $\mathbf{x} = 100111$ is

$$f(\mathbf{x}) = f(100111) = f(39) = (39 - 10)^2 = 841$$

1. Choose an initial population $P(0)$, usually by random.

The number N of chromosomes in $P(0)$ is called the *population size*.

e.g. Choose 4 chromosomes by random, for example

$$P(0) = \{ 101000, 100001, 101001, 001001 \} \quad (*)$$

The population size is $N = 4$.

2. During each iteration k , we operate on the chromosomes of $P(k)$.

We denote the elements of $P(k)$ by $\mathbf{x}_1^{(k)}, \mathbf{x}_2^{(k)}, \dots, \mathbf{x}_N^{(k)}$.

e.g. in (*), $\mathbf{x}_1^{(0)} = 101000, \mathbf{x}_2^{(0)} = 100001, \mathbf{x}_3^{(0)} = 101001, \mathbf{x}_4^{(0)} = 001001$

Next we compute the fitness values $f(\mathbf{x}_i^{(k)})$ of all members $\mathbf{x}_i^{(k)}$ of $P(k)$, and compute the *sum of fitness values*,

$$F(k) = f(\mathbf{x}_1^{(k)}) + f(\mathbf{x}_2^{(k)}) + \dots + f(\mathbf{x}_N^{(k)}) = \sum_{i=1}^N f(\mathbf{x}_i^{(k)})$$

$$\begin{aligned} \text{e.g. in (*), } F(0) &= f(101000) + f(100001) + f(101001) + f(001001) \\ &= f(40) + f(33) + f(41) + f(9) \\ &= 30^2 + 23^2 + 31^2 + (-1)^2 = 2391 \end{aligned}$$

We generate the next population $P(k+1)$ in two stages.

Selection stage: Form a new set of chromosomes $M(k)$ of size N , called the *mating pool*, from the chromosomes in $P(k)$ as follows:

Each element $\mathbf{m}^{(k)}$ in $M(k)$ is chosen to equal the element $\mathbf{x}_i^{(k)}$ of

$P(k)$ with probability $p_i = \frac{f(\mathbf{x}_i^{(k)})}{F(k)}.$

(So chromosomes in $P(k)$ of high fitness value are more likely to be selected for $M(k)$ than those with low fitness value. This is called the *roulette-wheel scheme*. There are also other schemes available: *tournament selection* and *rank selection*)

e.g. in (*), $M(0) = \{ \mathbf{m}_1^{(k)}, \mathbf{m}_2^{(k)}, \mathbf{m}_3^{(k)}, \mathbf{m}_4^{(k)} \}$

$$p_1 = \frac{f(\mathbf{x}_1^{(0)})}{F(0)} = \frac{30^2}{2391} = 37.64\% \quad \text{--- probability that } \mathbf{m}_i^{(k)} = 101000$$

$$p_2 = \frac{f(\mathbf{x}_2^{(0)})}{F(0)} = \frac{23^2}{2391} = 22.13\% \quad \text{--- probability that } \mathbf{m}_i^{(k)} = 100001$$

$$p_3 = \frac{f(\mathbf{x}_3^{(0)})}{F(0)} = \frac{31^2}{2391} = 40.19\% \quad \text{--- probability that } \mathbf{m}_i^{(k)} = 101001$$

$$p_4 = \frac{f(\mathbf{x}_4^{(0)})}{F(0)} = \frac{(-1)^2}{2391} = 0.0004\% \quad \text{--- probability that } \mathbf{m}_i^{(k)} = 001001$$

Evolution stage: Now we apply crossover and mutation operations.

Crossover: Choose and arrange some of the chromosomes in the mating pool $M(k)$ in pairs by random. (We can select just one pair, or several pairs. The probability for any chromosome to be selected is denoted by p_c)

The two chromosomes in each pair are called **parents**. Each parent pair exchanges substrings of their chromosomes as follows:

One-point crossover. Choose a number s between 1 and $L - 1$ by random (“crossing site”). The two parents then exchange their substrings to the left of the crossing site.

Two-point crossover. Perform two one-point crossovers.

The new chromosomes are called *offspring*. We replace the parents by their offspring in $M(k)$.

e.g. consider the chromosome pair 010001, 101000.

One-point cross-over: If $s = 3$

then the offspring are 101001 and 010000.

Two-point cross-over: If $s = 3$ and then $s = 2$

then the offspring are 011001 and 100000.

Mutation: For each chromosome $\mathbf{m}^{(k)}$ in $M(k)$, randomly change each symbol of the chromosome to another symbol with probability p_m . (p_m is

usually very small, e.g. <0.01 , so not every chromosome will change by mutation, and in those who change, only very few symbols will change.)

e.g. in (*), consider the chromosome 010001.

One mutation could change it to 010101 for example.

There could be two mutations to 000101 with very low probability.

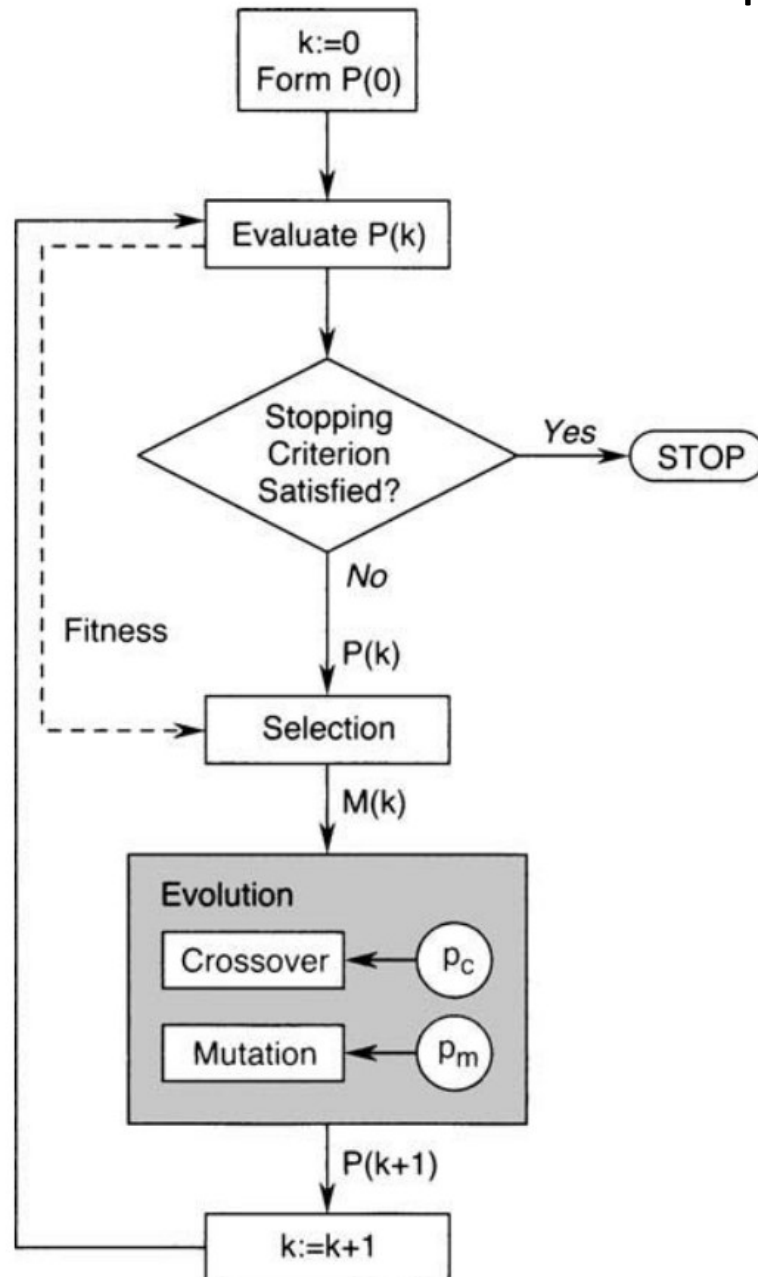
(so in a binary alphabet $\{0,1\}$, mutation means “flipping a bit”.)

After crossover and mutation, the mating pool $M(k)$ becomes the new population $P(k+1)$.

3. Repeat this procedure until the stopping criterion is satisfied. A stopping criterion could be
 - i) A prescribed number of iterations has been performed, or
 - ii) when the best fitness values don't improve by much any more

Of course, the best (highest) fitness value among all the N chromosomes will be the solution to the maximization problem.

Flowchart for the genetic algorithm



Exercise: Let us run ONE iteration of the genetic algorithm by hand for the above problem, when at the k -th iteration, **the** population $P(k)$ consists of the 4 chromosomes

$$\mathbf{x}_1^{(k)} = 101010, \quad \mathbf{x}_2^{(k)} = 011001, \quad \mathbf{x}_3^{(k)} = 101101, \quad \mathbf{x}_4^{(k)} = 001111$$

Sol:

1. Evaluate $P(k)$, i.e. compute the fitness values:

$$f(\mathbf{x}_1^{(k)}) =$$

$$f(\mathbf{x}_2^{(k)}) =$$

$$f(\mathbf{x}_3^{(k)}) =$$

$$f(\mathbf{x}_4^{(k)}) =$$

The sum of these is $F(k) =$

2. Construct the *mating pool* $M(k)$ using the roulette-wheel scheme.

The probabilities are:

$$p_1 = \frac{f(\mathbf{x}_1^{(k)})}{F(k)} =$$

$$p_3 = \frac{f(\mathbf{x}_3)}{F(k)} =$$

$$p_2 = \frac{f(\mathbf{x}_2^{(k)})}{F(k)} =$$

$$p_4 = \frac{f(\mathbf{x}_4^{(k)})}{F(k)} =$$

Now we construct cumulative probabilities:

$$c_0 = 0 \qquad c_1 = p_1 = \qquad c_2 = p_1 + p_2 =$$

$$c_3 = p_1 + p_2 + p_3 = c_2 + p_3 =$$

$$c_4 = p_1 + p_2 + p_3 + p_4 = c_3 + p_4 =$$

Pick a number r in $[0,1]$ by random (with the uniform distribution).

We choose $r =$. Because $\leq r <$ then $\mathbf{m}_1^{(k)} = \mathbf{x}^{(k)} =$

We choose $r =$. Because $\leq r <$ then $\mathbf{m}_2^{(k)} = \mathbf{x}^{(k)} =$

We choose $r =$. Because $\leq r <$ then $\mathbf{m}_3^{(k)} = \mathbf{x}^{(k)} =$

We choose $r =$. Because $\leq r <$ then $\mathbf{m}_4^{(k)} = \mathbf{x}^{(k)} =$

The mating population is

$$M(k) =$$

3. Evolution stage 1: *Crossover*. Choose one pair of chromosomes from $M(k)$ by random, and perform a one-point crossover.

Say, our choice results in the pair $\mathbf{m}^{(k)} =$ and $\mathbf{m}^{(k)} =$

Note: The probability that any of the 4 chromosomes in $M(k)$ is chosen is then $p_c =$

We choose randomly an integer s , $0 \leq s \leq L - 1 = 5$

Say, our choice is $s = 4$. Perform the crossover:

Replace the parents $\mathbf{m}^{(k)}$, $\mathbf{m}^{(k)}$ in $M(k)$ by these offsprings, to obtain

$$M(k) =$$

4. Evolution stage 2: *Mutation*. We choose a mutation probability of any symbol of $p_m = 0.01$.

Repeatedly, choose any (uniformly distributed) random number r in $[0,1]$. If $r \leq p_m = 0.01$, then “flip” the corresponding symbol.

$$\mathbf{m}_1^{(k)} = \quad : \quad r_1 =$$

$$\mathbf{m}_2^{(k)} = \quad : \quad r_1 =$$

$$\mathbf{m}_3^{(k)} = \quad : \quad r_1 =$$

$$\mathbf{m}_4^{(k)} = \quad : \quad r_1 =$$

Therefore,

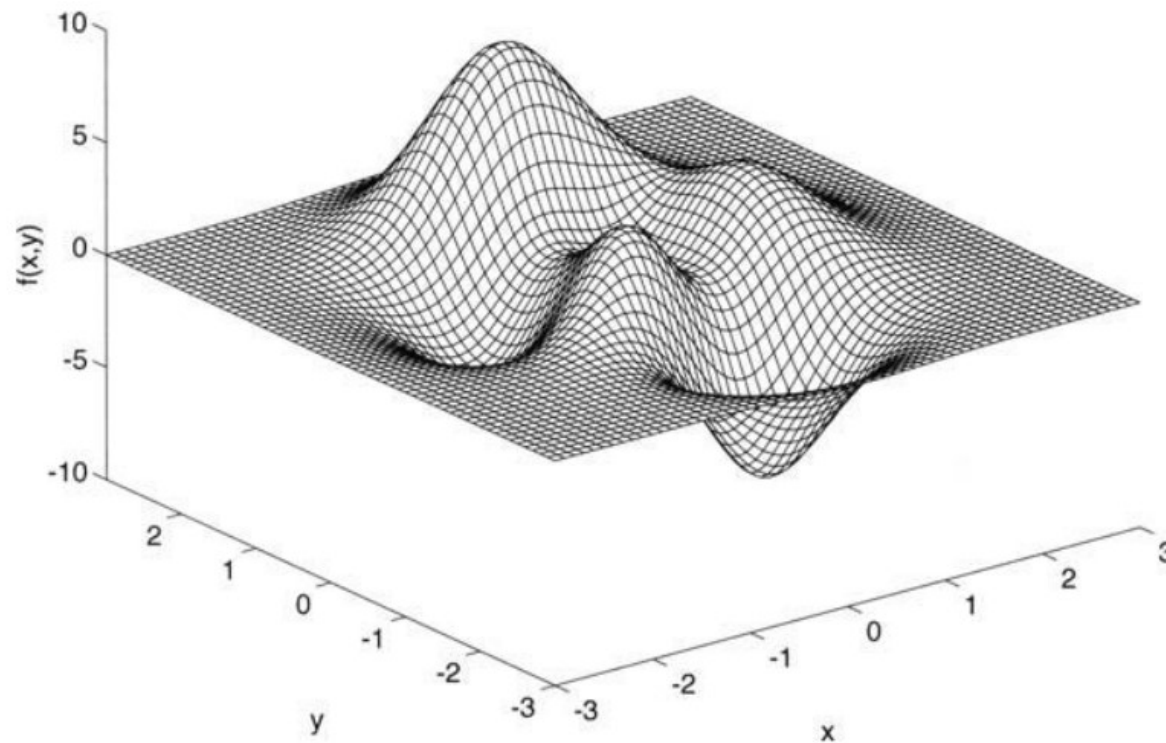
$$P(k+1) = M(k) =$$

Example: Consider the function $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ given by

$$f(x, y) = 3(1-x)^2 e^{-x^2-(y+1)^2} - 10\left(\frac{x}{5} - x^3 - y^5\right) e^{-(x^2+y^2)} - \frac{1}{3} e^{-(x+1)^2-y^2}$$

Maximize $f(x, y)$ over $\Omega = \{ (x, y) : -3 \leq x \leq 3, -3 \leq y \leq 3 \}$

Sol:



This function is too complicated to compute by hand. Using numerical methods, one can obtain:

f takes its max. value at approx. $(x, y) = (-0.0093, 1.5814)$,

and its max value is approx. $f(-0.0093, 1.5814) = 8.1062$

Let's use the genetic algorithm.

Step 1: Encode the points in Ω as chromosomes. Problem: Ω contains infinitely many points, but we can only have finitely many chromosomes (i.e. points from Ω). We choose 16 bits for the x -coordinates, and 16 bits for the y -coordinates, so a chromosome has length $L = 32$.

We place a grid of $2^{16} \times 2^{16}$ equally spaced points over Ω as follows:

$$\text{Set } \tilde{x} = (x + 3) \frac{2^{16} - 1}{6} \quad \text{and} \quad \tilde{y} = (y + 3) \frac{2^{16} - 1}{6} \quad (**)$$

(i.e. we translate + scale the square Ω).

Then we have $0 \leq \tilde{x}, \tilde{y} \leq 2^{16} - 1$ for all $(x, y) \in \Omega$. That is

$$(\tilde{x}, \tilde{y}) \in [0, 2^{16} - 1] \times [0, 2^{16} - 1] \quad (2^{16} - 1 = 65535)$$

Now encode the points $(\tilde{x}, \tilde{y})$ that have integer coordinates in binary form.

(e.g. The point $(x, y) = (-1, 3) \in \Omega$ becomes the point $(\tilde{x}, \tilde{y})$ where

$$\tilde{x} =$$

$$\tilde{y} =$$

which is then encoded as $\underbrace{0101010101010101}_{x=-1 \text{ encoded}} \underbrace{1111111111111111}_{y=3 \text{ encoded}}$

We can go back from $(\tilde{x}, \tilde{y})$ to (x, y) by solving (**):

$$x = \frac{6}{2^{16}-1} \tilde{x} - 3 \qquad y = \frac{6}{2^{16}-1} \tilde{y} - 3 \qquad (***)$$

(This is the way to compute the fitness of a chromosome in the algorithm: Convert a chromosome string back to a decimal number $(\tilde{x}, \tilde{y})$, and then substitute (***) into $f(x, y)$)

Running the genetic algorithm with

- population size $N = 20$
- 200 iterations
- $p_c = 0.8$ (i.e. the probability that any chromosome is selected for crossover)
- $p_m = 0.0075$ (probability of mutation of a symbol/gene)
- Keep the “best” chromosome in each iteration

We have obtained in 5 runs of the algorithm:

Run 1: $(x, y) = (0.0080, 1.5673)$, max value $f(x, y) = 8.1004$.

Run 2: $(x, y) = (-0.0099, 1.5863)$, max value $f(x, y) = 8.1058$.

Run 3: $(x, y) = (-0.0190, 1.6022)$, max value $f(x, y) = 8.0984$.

Run 4: $(x, y) = (-0.0098, 1.5807)$, max value $f(x, y) = 8.1062$.

Run 5: $(x, y) = (1.5005, 0.0508)$, max value $f(x, y) = 3.2562$.

Note that every run of the algorithm will give a slightly different result because of the many random choices.

In some runs of the algorithm the optimal solution could not be found.

There are numerous variations to the genetic algorithm to try to improve its performance.

Modification: Real-Number Genetic Algorithms.

Because there are only finitely many chromosomes available, the genetic algorithm used above can only find the max of the objective function $f(x, y)$ on a discrete subset of Ω , usually a grid. Ω

One can modify the algorithm to make it “genetic-like”, operating on all of Ω . So a “chromosome” is now a vector with real components.

Crossover: One can modify the cross-over step in several ways.

- a) For parents x, y **one** offspring is $z = (x + y) / 2$. It can replace one of the parents, or
- b) one can obtain **two** offsprings

$$z_1 = (x + y) / 2 + w_1 \quad z_2 = (x + y) / 2 + w_2$$

where w_1 and w_2 are randomly generated numbers with zero mean.

(if z_1 or z_2 are outside of Ω , then modify to bring back inside Ω .)

- c) Choose a number $\alpha \in (0,1)$ by random, and obtain two offsprings

$$z_1 = \alpha x + (1 - \alpha)y \quad z_2 = (1 - \alpha)x + \alpha y$$

- d) Combine c) and d): Choose a number $\alpha \in (0,1)$ by random, and obtain two offsprings

$$z_1 = \alpha x + (1 - \alpha)y + w_1 \quad z_2 = (1 - \alpha)x + \alpha y + w_2$$

with w_1 and w_2 randomly generated numbers with zero mean.

Mutation: One can also modify the cross-over step in several ways.

- a) Add a random vector to a “chromosome” x :

$$x_{new} = x + w, \quad w \text{ randomly generated with zero mean.}$$

- b) Choose a random number $\alpha \in (0,1)$ and a random point $w \in \Omega$, and set $x_{new} = \alpha x + (1 - \alpha)w$

Example: Continue the previous example:

$$f(x, y) = 3(1-x)^2 e^{-x^2-(y+1)^2} - 10\left(\frac{x}{5} - x^3 - y^5\right) e^{-(x^2+y^2)} - \frac{1}{3} e^{-(x+1)^2-y^2}$$

Maximize $f(x, y)$ over $\Omega = \{ (x, y) : -3 \leq x \leq 3, -3 \leq y \leq 3 \}$

We choose:

Running the genetic algorithm with

- population size $N = 20$
- 200 iterations
- $p_c = 0.8$ (i.e. the probability that any chromosome is selected for crossover)
- $p_m = 0.0075$ (probability of mutation of a symbol/gene)
- Keep the “best” chromosome in each iteration
- Choose “crossover” method d)
- Choose “mutation” method b)

We have obtained in 5 runs of the algorithm:

Run 1: $(x, y) = (0.0223, 1.5882)$, max value $f(x, y) = 8.1040$.

Run 2: $(x, y) = (-0.0457, 1.5701)$, max value $f(x, y) = 8.0935$.

Run 3: $(x, y) = (-0.0432, 1.6042)$, max value $f(x, y) = 8.0880$.

Run 4: $(x, y) = (0.0195, 8.0984)$, max value $f(x, y) = 8.0984$.

Run 5: $(x, y) = (-0.0190, 1.5782)$, max value $f(x, y) = 8.1053$.